

EXACT 2026 Technical Solution Brief

Team: AI WITH BRO | **System:** Open-source symbolic reasoning API for logic and physics QA

The core design choice is to treat each query as a constrained reasoning problem, not as free-form chat generation. The LLM proposes formal executable structure; Z3 or SymPy performs the decisive computation; the final explanation is grounded in the executed trace.

Architecture Positioning

The submitted service exposes one FastAPI endpoint, `/predict`, for the unified EXACT 2026 test stream. The classifier first honors any explicit `task_type` or `query_type` field. If no explicit type is provided, it uses the official schema signal: non-empty premises-NL routes to Type 1 logic, while empty premises route to Type 2 physics. This keeps routing deterministic and avoids spending model budget on a task that can be inferred from the input contract.

Reasoning Pipeline

Stage	Engineering role	Reliability control
Classifier	Selects logic or physics path without an LLM call.	Deterministic schema-based routing.
Retrieval	For physics, retrieves formulas and worked examples from disclosed corpora.	Hybrid BM25/vector search with reranking.
Formalizer	Generates Z3 code for logic or SymPy code for physics.	Self-hosted open-source coder model; no external LLM API.
Solver	Executes the generated program and returns the symbolic/numeric result.	Subprocess isolation, timeout budget, and one execution-feedback retry.
Explainer	Converts the solver result into the official response schema.	Explanation is conditioned on code, premises, retrieved evidence, and solver output.

Evaluation Alignment

The system is optimized around the three scoring dimensions. For **P1 correctness**, answer selection is anchored to symbolic execution rather than unconstrained text. For **P2 explanation quality**, the response explains the applicable law, premise chain, or calculation path in natural language. For **P3 reasoning depth**, the API returns structured evidence fields: `fol`, `cot` as a concise derivation summary, premises, and calibrated confidence. The API sanitizer enforces valid output types, clamps confidence to `[0, 1]`, and guarantees non-empty mandatory answer and explanation fields.

Models and Serving

- **Qwen2.5-Coder-7B-Instruct, fine-tuned:** formal program generation for Z3 and SymPy.
- **Qwen2.5-7B-Instruct, fine-tuned:** final structured explanation and JSON response generation.
- Models are served locally through `llama.cpp` / `llama-server`. The HTTP interface is OpenAI-compatible only at the protocol level; it does not call OpenAI, GPT, Claude, Gemini, or any commercial model endpoint.

Data and Compliance

All training, retrieval, and evaluation sources are disclosed in the Data Disclosure Document. The pipeline does not use closed-source LLMs for training data generation, preprocessing, retrieval, evaluation, or inference. If the internally digitized Electro textbook set cannot be accompanied by a complete bibliography, it must be excluded from training, RAG, and final submission artifacts.